

Your Cheat Sheet for Getting the Most Out of Every Session

Based on your personalised 20-week Frontend Mastery + Programmer Fundamentals roadmap · Angular + TypeScript · 1.5–2 hrs/day · Depth over speed

// 01

When You Don't Understand a Concept

Use these when a topic feels abstract or you've read the docs and it still doesn't click.

UNDERSTAND THE WHY — not just the what

- Explain [concept] like I'm about to build something with it, not study it for an exam
- Why does [concept] exist? What specific problem does it solve that nothing else could?
- What was Angular like before [concept] existed? What pain did it remove?
- Give me the simplest real-world Angular example of [concept] — no to-do apps

CLARIFY CONFUSION — when two things look the same

- What is the actual difference between [X] and [Y] — and when would I choose one over the other in a real project?
- I keep confusing [X] and [Y]. Can you give me a concrete side-by-side comparison with code?
- Is [concept] the same as [other concept] I already know? How are they related?
- In what situation would using [X] be a mistake? What should I use instead and why?

SIMPLIFY — when feeling overwhelmed by a topic

- Strip [concept] down to the 20% I need to build something today. What can I skip for now?
- Explain [concept] in one paragraph, plain English, no jargon
- What are the 3 things about [concept] I absolutely must understand before anything else?
- Give me a mental model for [concept] — an analogy or image that makes it stick

// 02

When Your Code Isn't Working

Never paste code and say 'fix it'. Use these prompts to debug like a senior developer.

DEBUG TOGETHER — don't just get the answer

- Here's my code and here's the error: [paste both]. Walk me through HOW to debug this — don't just give me the fix
- What are the 3 most likely reasons this isn't working? Let me test each one.
- I expected [X] to happen but [Y] happened instead. What does that tell us about the bug?
- What console.log or breakpoint would you add first to isolate this problem?

CODE REVIEW — make your code better, not just working

- Review this code and tell me what a senior Angular developer would change — and explain the why behind each change
- What are the code smells or anti-patterns in this snippet? [paste code]
- How would you refactor this to follow SOLID principles? Show me the before and after.
- Is there a cleaner or more idiomatic Angular way to do what I'm doing here?
- What edge cases or failure modes am I not handling in this code?

// 03

When You Feel Overwhelmed or Stuck on a Phase

Use these to reset, refocus, and get moving again without spiralling.

RESET & REFOCUS — when everything feels like too much

- I'm on Phase [X] of my roadmap and feeling overwhelmed. What is the single most important thing to focus on right now?
- I've been stuck on [topic] for [X] days. What's likely going wrong with my approach?
- I feel like I'm not making progress. Help me see what I've actually learned vs where I started.
- What can I safely skip in [topic] for now, learn later, and still build something meaningful today?

UNSTICK — when you've been staring at the same problem too long

- I've been stuck on [specific problem] for [X] minutes. Here's what I've tried: [list]. What would you try next?
- Give me a completely different approach to solving [problem] — forget what I've been trying
- Is [assumption I'm making] actually correct? Am I solving the wrong problem?
- What's the smallest possible version of this that could work? Let me get that working first.

// 04

When You Want to Go Deeper

Use these after you've built something and want to level up your understanding.

LEVEL UP — push past working to understanding

- I understand [X] and can use it. What's the next level of that concept I should learn now?
- How would a senior Angular developer approach [problem] differently than I just did?
- I got this working — but I don't fully understand why it works. Explain the mechanics.
- What would make this code production-ready? What am I missing that matters at scale?
- What does Angular do under the hood when I call [API/method]? What's actually happening?

CONNECT THE DOTS — build mental models across topics

- How does [topic A] relate to [topic B]? Where do they overlap or interact?
- Now that I understand [X], how does it change the way I should think about [Y]?
- Show me a scenario where knowing [concept] prevents a real bug or performance issue
- What patterns or mistakes do developers commonly make with [concept] that I should be aware of?

// 05

For Interview Preparation (Phase 5)

Use these in the final 3 weeks. Practice speaking, not just thinking.

MOCK INTERVIEW — get quizzed and get feedback

- Ask me a senior Angular interview question about [topic] and give me detailed feedback on my answer
- What's the tricky follow-up question an interviewer would ask after I say [X]?
- Roleplay as a tough technical interviewer for a senior frontend role. Start with system design.
- I answered [topic] with [my answer]. What would a strong answer sound like? What did I miss?
- Give me 5 Angular questions that separate junior from senior candidates — then quiz me

WAR STORIES — turn experience into interview gold

- Turn this experience into a STAR-format answer: [brief description of what I built or fixed]
- How do I make my answer about [experience] sound more senior and specific?
- What quantifiable results or metrics would make my story about [X] more compelling?
- I want to talk about [topic] in interviews but I'm not sure how. Help me frame it professionally.

SYSTEM DESIGN — practice thinking out loud

- Walk me through how you'd design [feature/app] in Angular from scratch — then critique my own answer
- I need to pick a state management strategy for [type of app]. Help me reason through the tradeoffs.
- How would I structure a large Angular monorepo with multiple feature teams?
- What questions should I ask the interviewer when given a frontend system design prompt?

// 06

Daily & Weekly Habit Prompts

Use these at the start or end of each session to stay structured and reflective.

SESSION STARTERS — know exactly what to do before you open VS Code

- I have 90 minutes today. I'm on Week [X] of Phase [Y]. What should I build or practice right now?
- Give me a focused 60-minute challenge using [concept] that adds to my existing project
- I only have 30 minutes today. What's the highest-value thing I can do that still moves the needle?
- I want to practice [concept] hands-on. Give me a small, concrete task I can complete in one session.

WEEKLY REVIEW — every Friday, run through these

- I learned [list topics] this week. Quiz me on each one to see if it actually stuck.
- I built [X] this week. What should I refactor or improve before moving to next week?
- What did I learn this week that I could apply directly to my current work project on Monday?
- I'm still unclear on [topic]. Give me one concrete exercise to resolve that before the weekend.
- Am I ready to move to the next phase? Quiz me on the Phase [X] exit criteria.

// 07

Programmer Fundamentals Prompts

Keep these fundamentals sharp throughout all phases — not just Phase 0.

CLEAN CODE & SOLID — apply to real code daily

- Review this function and tell me which SOLID principles it violates — then show me the fix
- Is this the cleanest way to write [X]? Show me what Robert Martin's Clean Code version looks like.
- I have a function doing [X]. How do I apply Single Responsibility to break it apart?
- What code smells exist in this snippet and how would you refactor them? [paste code]

DSA & PROBLEM SOLVING — practical, not academic

- I need to [task involving data]. What's the most efficient data structure for this and why?
- What's the Big O complexity of what I just wrote? Is there a faster approach?
- Walk me through how to decompose [complex problem] step by step before writing any code
- I need to traverse/search [hierarchical data]. Help me think through the recursive approach.

DEBUGGING — systematic, not random

- Teach me how to debug [type of bug] systematically using Chrome DevTools
- What's the best way to reproduce this bug reliably before trying to fix it?
- I'm getting [error type]. Walk me through reading this stack trace top to bottom.
- My Angular app is slow. Walk me through a systematic performance investigation from scratch.

// QUICK REFERENCE

The 7 Most Powerful Prompt Patterns — Memorise These

SITUATION	I'm on Phase [X], working on [Y], and I'm stuck on [Z].
BUILD MODE	Don't just explain it — give me a task that forces me to use it.
WHY FIRST	Why does [concept] exist before you show me how to use it.
TRADEOFFS	What are the tradeoffs of [X] vs [Y] in a real enterprise context?
FEEDBACK	Here's my code / answer — what would you change and why?
SIMPLIFY	Strip this to the 20% I need. What can I safely skip for now?
NEXT LEVEL	I understand X — what should I learn next that builds on it?

PRO TIP

Always give Claude context: your current phase, what you just built, and what specifically isn't working or clicking. The more specific your prompt, the more targeted and useful the response. "I don't understand RxJS" gets a generic answer. "I'm building the Task Manager in Phase 1 and switchMap is cancelling requests I don't want cancelled" gets a precise, actionable one.